

Laboratorio 07 – Procesos con fork()

Objetivos

Aplicar de forma práctica los conceptos de creación de procesos mediante la función `fork()`, incluyendo jerarquías, control de procesos zombi.

Instrucciones

Esta actividad se realizará individualmente. Subir un archivo `.cpp` para cada ejercicio planteado, con código comentado (incluir encabezado), ordenada, modular y funcional, debe ser ordenado e identificado en cada actividad.

Recuerda para compilar y ejecutar programas en WSL o similares:

- **Compilar programas en C:** `gcc programa.c -o programa`
- **Compilar programas en C++:** `g++ programa.cpp -o ejecutable`
- **Ejecutar:** `./ejecutable`

Ejercicios

1. 5 pts. Analiza el código `fork_71.cpp`. Explica lo siguiente:

- 3 pts ¿Cuántas veces se solicita que se ingrese el valor de la variable `a`? ¿Explica por qué se realiza esa cantidad de veces la solicitud de ingreso de `a`?
- 2 pts Después de llamar a la función `fork()`, ¿cuántas variables con el nombre `a` existen en el address space de ejecución del programa? ¿por qué?

2. 15 pts. Analiza el código `fork_72.cpp`. Explica lo siguiente:

- 3 pts ¿En ambos ejemplos se generan procesos hijos paralelos?
- 5 pts Dibuja un diagrama que muestre cómo se generan los procesos hijos para este programa.
- 3 pts ¿Qué sucede cuando un hijo llama a `exit`?
- 2 pts ¿Qué sucede cuando el proceso padre llama a `wait`?

3. 10 pts. Investiga las siguientes funciones:

FUNCIÓN	DESCRIPCIÓN DETALLADA: 1. Uso principal 2. Parámetros 3. Valor de retorno 4. Ejemplo de uso
<code>execvp()</code>	
<code>sleep()</code>	
<code>wait()</code>	
<code>kill()</code>	
<code>exit()</code>	

4. 10 pts. Analiza el código fork_73.cpp. Explica lo siguiente:

- 3 pts Dibuja el árbol de procesos generado por las dos llamadas a fork(). Indica qué procesos son creados en la primera llamada y cuáles en la segunda.
- 3 pts Al habilitar la tercera llamada a fork(), determina el número total de procesos generados y dibuja el nuevo árbol.
- 4 pts Explica cómo el número de llamadas a fork() influye en la cantidad total de procesos generados. Justifica tu respuesta describiendo el patrón de crecimiento observado.

5. 10 pts. Analiza el código primesFork.cpp. Realiza lo siguiente:

Analiza la entrada del programa y modifica el código para que el exponente sea leído como un valor entero, en lugar de procesarlo como un carácter ASCII. Explica por qué esta mejora hace más robusto el programa.

- 5 pts Incorpora el uso de wait() en el proceso padre. Luego explica qué problema puede ocurrir si el padre no espera la finalización del hijo.
- 5 pts Revisa la función isPrime() y explica por qué su implementación no es eficiente para valores grandes. Mejora la función para que también maneje correctamente casos inválidos o no considerados explícitamente, como números negativos. Explica qué cambio realizaste.

6. 50 puntos. Desafío: crea tu primer compilador paralelo.

En este desafío, desarrollarás un programa en C++ que simule un decodificador de instrucciones de 7 bits, con la particularidad de que cada instrucción será procesada por un proceso distinto usando fork().

- a) El programa solicitará al usuario una secuencia de múltiples instrucciones binarias (mínimo 3 cadenas de 7 bits separadas por espacios, por ejemplo:

1100110 0011100 1001101.

- b) Cada instrucción se divide en partes, donde cada grupo de bits tiene un significado específico. La estructura es la siguiente:

[3 bits OPCODE] [2 bits OPERANDO A] [2 bits OPERANDO B]

Esto significa que:

- Los primeros 3 bits (bits 6–4) indican la operación que se debe realizar
- Los últimos 4 bits se dividen en dos operandos de 2 bits cada uno

OPCODE (bits 7-5)	Operandos A y B (A bits 3-2) (B bits 1-0)
000 → Suma	00 → 0
001 → Resta	01 → 1
010 → Multiplicación	10 → 2
011 → División entera	11 → 3
101 → Potencia (A^B)	
110 → A mod B	

Para la instrucción: 101 10 01	
Se interpreta así: - OPCODE = 101, que representa potencia (A^B) - A = 10 en binario, equivale a 2 en decimal - B = 01 en binario, equivale a 1 en decimal	Resultado: 2^1 = 2

c) Distribución de procesos con fork():

- Si se ingresan n instrucciones válidas, el programa debe crear exactamente $n - 1$ procesos hijo usando la llamada al sistema `fork()`.
- La ejecución de las instrucciones se reparte así:
 - **Proceso padre:** ejecuta la primera instrucción.
 - **Primer hijo creado:** ejecuta la segunda instrucción.
 - **Segundo hijo creado:** ejecuta la tercera instrucción, y así sucesivamente.
- Cada proceso (padre o hijo) debe imprimir:
 - Su PID.
 - Su número de instrucción ejecutada (1, 2, 3...).
 - El resultado de la operación decodificada.
 - Tiempo que tardó en ejecutarse el proceso.
- Los procesos hijos deben finalizar con `exit()`, mientras que el padre debe esperar a todos sus hijos antes de terminar (usando `wait()`).

```

Ingresa instrucciones binarias (separadas por espacios): 01011011 01101110 01110010
Proceso padre ejecutando instruccion 1, PID: 975
Instruccion 1: 0101011
  OPCODE: 2 (Multiplicacion)

  A: 2
  B: 3
  Resultado: 6

PID: 975 - Tiempo de ejecucion: 88 microsegundos.

Proceso hijo ejecutando instruccion 2, PID: 979
Instruccion 2: 01101110
  OPCODE: 3 (Division)
  :
  A: 3
  B: 2
  Resultado: 1

PID: 979 - Tiempo de ejecucion: 278 microsegundos.

Proceso hijo ejecutando instruccion 3, PID: 980
Instruccion 3: 0111000
  OPCODE: 3 (Division)

  A: 2
  B: 0
  Resultado: Error. Division por cero

PID: 980 - Tiempo de ejecucion: 433 microsegundos.

```

Material adicional por entregar: video explicativo, demostrando la ejecución del programa realizando las siguientes pruebas:

- Instrucciones válidas y resultados correctos
- Generar errores intencionales: división por cero, error por ingreso de caracteres diferentes de 0 y 1, una instrucción inválida (menos de 7 bits o caracteres incorrectos).
- Explicar todas las funciones y/o librerías nuevas e implementadas en el código, y justificar por qué es necesario incluirlas en esta versión del programa.
- Explicar por qué los procesos hijos deben finalizar con `exit()`, mientras que el padre debe esperar a todos sus hijos antes de terminar (usando `wait()`).
- Si utilizas algún tipo de estructura cíclica dentro de la cual creas a los procesos hijo con `fork()`, ¿realmente se crean procesos hijos paralelos?